Beavgredients Project

By: Levi Cook, Kenneth Robertson, Nico Sarmiento, Alexander Sahlstrom

Last Edited: 1/19/26 10:11 PM - Levi + Alex

Team Info

Levi Cook : ***Backend developer***

- Experience with database management in web development
- Familiarity with Node.js and API integration

Kenneth Robertson : ***Manager / QA Lead***

- Experience with managing teams
- Understanding of AGILE flow and vision for the project.

Nico Sarmiento : ***front end developer / designer***

- Experience with React.js, Next.js
- Familiarity with Figma to design front end

Alexander Sahlstrom : ***System Architect / DevOps Engineer***

- Strong understanding of database design and front-backend communication.
- Experience with Node.js, javascript, and SQL

https://github.com/robekenn/Beavgredients

https://docs.google.com/presentation/d/1Hoj4quZlw56vGgz32zJsYPgI7zhypSYY_H5p2gS5tOk/edit?usp=sharing

Using React.js Next.js & Node.js

Discord, Instagram

Product Description

Abstract

Knowing what recipes you can cook any given day requires you to know what ingredients you have on hand. Beavgredients is a web application that recommends recipes to users given what ingredients they have on hand. Additionally, if a user wants more options, they can explore recipes that require more ingredients and can add necessary ones to a curated grocery list. Beavgredients provides an easy to use tool for anybody trying to eat new foods but doesn't know where to start.

Goal

This application aims to help users who are looking to save time when cooking and discover new recipes based on ingredients that are already in their home while also saving them costs on groceries. It does this by taking into consideration what ingredients the user already has and suggests meals that the user could create, it then compiles a grocery list for any missing ingredients.

Current Practice

Apps have the list of ingredients generating possible recipes, but when trying to recommend ingredients you don't have they lack depth. They don't explain why they recommend certain ingredients, and they end up unlocking 1000 new variations of banana bread. Additionally, they don't allow for a user to generate a shopping list if they want to make recipes that require more ingredients than what they have input.

Novelty

After some research we quickly discovered that there are many applications that have similar features to our project. One such application is SuperCook which can display meal recipes based on the ingredients or even the diet you select. You can favorite meals you like and save ingredients to your accounts pantry. Beavgredients will have all of these features but unlike SuperCook, our application will also allow you to generate

shopping lists that can be sent to your email. Users can also create and add custom recipes to their account, these can also be shared with friends.

Effects

Beavgredients would make sure that no ingredient goes to waste by providing multiple ways to use that ingredient in meals. If a user wants to start cooking but doesn't know where to start, Beavgredients is an easy way to start exploring recipes you can make at home. Finally, it can make shopping easier and remove indecision as Beavgredients will curate a shopping list of all the ingredients you need to make the meals you want.

Technical Approach

Currently, we plan on using Next.js to handle our frontend development. For our backend development, we are going to use Node.js. Finally, to access different recipes, we plan on using TheMealDB API.

Features

Major Functions:

1. Generates List of Recipes given Ingredients
2. Grocery list Compiler and Emailer
3. User Log-in/account creation
4. Searching Recipes
    a. Filter
        i. Cuisine
        ii. Restrictions
        iii. Type of Meal
        iv. Use Specific Ingredient
5. Creating custom recipes

Minor Functions:

1. Recommend an ingredient that opens the door to the most recipes
2. Tells you cheapest place to buy ingredients in Corvallis area

---

Functional Requirements

- **Use Case 1** (Levi Cook)
    - A user will want a grocery list containing any ingredients they need for chosen meals
    - Pre-conditions: They have an account with Beavgredients and are logged into the web application, they have added meals to their “cart” that contain ingredients they don’t already have
    - Post-conditions: The user will receive an email containing an easy to read list of ingredients they need (A grocery list).
    - Steps:
        i. User logs in or creates an account
        ii. They select recipes they’d like to make and add them to their cart
        iii. They check out these meals
        iv. An email is sent to their associated account email containing the recipes and a grocery list
    - Extensions and Exceptions
        i. The meals they chose use ingredients they already have
            - The grocery list in the email will be empty
        ii. They don’t have an account/aren’t logged in
            - An email will be unavailable since the user hasn’t set up an email account
- **Use Case 2** (Alexander Sahlstrom)
    - **Actors:** User, TheMealDB
    - **Trigger:** The user is lactose intolerant and wants to avoid recipes that contain dairy products. However, they are having difficulty discovering new recipes.
    - **Pre-Conditions:** User is logged in and at the Beavgredients homepage.
    - **Post-Conditions:** The user will now have non-dairy meals saved to their favorites list on their account.
    - **Steps:**
        i. The user filters the meal catalogue by “non-dairy”
        ii. They then browse the catalogue favoriting meals they would like to try in the future.
        iii. The user then chooses a meal and generates a shopping list, exporting the list as a pdf and sends it to themselves via email.
        iv. The user then logs out or closes their browser tab.
    - **Extensions:**
        i. The user does not need to filter by ingredients to browse meals.
        ii. The user does not need to log in to export recipes.

    - **Exceptions:**
        i. The user cannot favorite meals if they are not logged into their account.
- **Use Case 3** (Kenneth Robertson)
    - **Actors:** User
    - **Trigger:** The user received a cook book from a family member but wants to keep them more organized and not have to write up ingredient lists by hand.
    - **Pre-Conditions:** The user has an account with Beavgredients and is in the homepage.
    - **Post-Conditions:** The user's custom meal recipe is now saved to their account and a grocery list can be generated in the future.
    - **Steps:**
        i. The user goes to their account profile page.
        ii. The user goes to their "bookshelf" where they can view their custom recipes.
        iii. They click the create new recipe button.
        iv. They enter a recipe name, ingredients, and a short description.
        v. The user then saves the recipe.
        vi. They then move the new recipe into their "breakfast" cookbook.
    - **Extensions:**
        i. The user could choose to delete a custom recipe or discard the changes they made.
        ii. The user could choose to not move the recipe into a cookbook folder.
    - **Exceptions:**
        i. The user cannot create custom recipes without an account.

- **Use Case 4** (Nico Sarmiento)
    - **Actors:** Users
    - **Trigger:** The user bought bulk ingredients and has leftover ingredients they want to use before they go bad.
    - **Pre-Conditions:** The user has an account with Beavgredients.
    - **Post-Conditions:** The user has recipes recommended to them with what ingredients they have on hand
    - **Steps:**
        i. The user inputs what ingredients they have at home
        ii. The user uses the filter to only show recipes using the ingredient they wish to use
        iii. The website recommends them recipes using that ingredient

- **Extensions:**
    i. The website will recommend recipes using ingredients recently added instead of needing to filter for it specifically
    ii. The user doesn't need to sign in to add to pantry, it'll forget after closing out
- **Exceptions:**
    i. User cannot save ingredients added to pantry without being logged in

Non-Functional Requirements

- User passwords are encrypted and securely stored
- User should be able to create an account in under a minute
- Search times should be under 2 seconds

External Requirements

- Any buttons that the user clicks on will function correctly/work as expected. Any navigation to a new page will correctly navigate to the expected page. Any text input (User login, meal search) will handle input as expected.
- The application will have a URL that can be accessed by anybody as long as the URL is correct
- When creating the Web App, all source code will be documented so that anybody who wasn't part of the dev team could easily set up the application or can make adjustments knowing what each portion of code/file does.
- The application will be of a grand enough scale to reflect the team of 4 developers that are working on it. I.E. the application will be polished, have a clean and easily understandable UI, and have features that are robust enough to reflect the size of the dev team and the time they had to complete the application.

Team Process Description

Describe your quarter-long development process.

- **Our development tools:**

    - We are developing a web application so we will be using react.js and next.js for front end and node.js for backend. Developing in one language is a huge advantage of using these tools.
    - We want to focus on the development of user features so for data we are planning to use an API call to TheMealDB, a database full of nearly 600 recipes.
- **What are our team roles and why:**
    - Levi - Backend developer
        - The backend is a key part of our product and we need someone to develop the backend systems of our application to bring our product's core functionality to life.
    - Alex - Systems Architect
        - A systems architect will be responsible for designing the interactions between the frontend and backend to make sure that they are compatible.
    - Kenneth - Manager & Quality Assurance
        - A manager will be responsible for keeping the team on schedule ensuring that we meet our milestones.
    - Nico - Frontend Developer
        - The frontend is a key aspect of our product and what users will be interacting with, having a frontend developer is integral to a good web app design.
- **Schedule:** **https://docs.google.com/spreadsheets/d/1lXsMWypnZNUypA2y_E22jH8DgNJV2eQ1mvpLWZyT9p8/edit?usp=sharing**
- **Risks:**
    - If one or multiple members of the team do not complete tasks it will result in our team falling behind and rushing production. This could lead to errors that could make our product unusable.
    - If we cannot find a database that is free to use or it is not well structured then this could lead to features being difficult or impossible to implement.
    - If we do not have a clear vision for our product it will make it difficult to develop as a team and features may become incompatible or unusable leading to failure.
- **Feedback**:
    - The most useful time to get feedback for our project is now during our design phase and whenever we are developing a new feature. This will allow us to easily make changes before things become too complex. We received feedback during our pitch and in the future we plan to receive

feedback from friends, colleagues and family to ensure a high quality product.

Risk Assessment

ID	Date Raised	Risk Description	Likelihood of risk Occuring	Impact if risk occurs	Serverity	Owner	Mitigating action
1	1/15/26	Data Inaccuracy	Medium	High	High	Front End Developer	Develop clean testable code that allows for faster traceability & debugging
2	1/15/26	Scalability & Performance	Low	Medium	Medium	Backend Developer	Develop modular and object oriented code.
3	1/15/26	Data Security	High	Low	Medium	Backend Developer	Ensure that user data is encrypted before being transferred and is stored in the backend.
4	1/15/26	Tasks Incomplete	Low	High	High	Team Issue	Start development early & meet deliverables within our timeline.
5	1/15/26	Bugs in Backend	Low	High	High	Backend Developer	Have the QA Manager run extensive testing.

Data Inaccuracy:

- **Evidence:** Data Inaccuracy is a very common problem in the field. Having protocol in the development can help us catch this.

- **Detection Plan:** We plan to run a series of tests that have been administered by the QA Manager.

Scalability & Performance:

- **Evidence:** One of the biggest challenges for small startups is the thought of scalability. We have chosen to deploy our web application using a free host since it is still a proof of concept but as we take steps to enter the market we plan to transition our application to a dedicated hosting service.
- **Detection Plan:** We will keep eyes on where our delays are taking place, if they go above a certain threshold we will deploy a new host with better performance.

Data Security:

- **Evidence:** In many projects there are a lot of ways for threats to overtake.
- **Detection Plan:** We will run automated front end tests before deployment to find any weaknesses in our architecture.

Tasks Incomplete:

- **Evidence:** Some features may be more complex than we initially anticipated which can lead to a growing backlog and missed deadlines..
- **Detection Plan:** If we fall behind on deadlines we will break tasks up into more feasible goals to ensure we can deliver our product on time. .

Bugs in Backend:

- **Evidence:** Backend bugs often occur within the products of new startups and software development in general, with this in mind we are able to seek ways to track down these bugs.
- **Detection Plan:** We will run a series of automated tests on the backend systems to detect any unexpected bugs that involve improper data retrieval and handling, as well as communication with the frontend of our product.

Project Schedule

Week #	Goals
1	● Establish our group ● Brainstorm project Ideas
2	● Create the requirements (Functional and Non-Functional) ● Decide the tools we will be using to develop our project ● Set up a GitHub repository for the project
3	● Design how the web application should look and how it should be formatted ● Solidify our project Idea
4	● Design how the web application should look and how it should be formatted ● Decide on a design architecture for our project ● Solidify our project Idea
5	● Begin Development this week ● Begin working on front end/UI ● Begin Backend development (Database and MealDB API integration) ● Implement recipe search features ● Implement Users adding to their ingredient list
6	● Test Week 5's features ● Finish Front end programming and design ● Implement email features ● Implement adding recipe features
7	● Test email features ● Test user interactions with the UI ● Test adding, removing, or searching recipe features ● Begin overall testing and debugging of the web app
8	● Begin having non developers test the software ● Make fixes accordingly
9	● Prepare for final tests and presentations
10	● Release week

Test Plan & Bugs

We plan to perform extensive testing on a number of different parts of our web application. This is integral to ensuring our users have positive experiences with our product.

Frontend UI:

- Each screen will have user tests to see if they can try to break the system by doing random actions such as adding and removing items from their cart, pantry, or favorites.

Backend:

- For each core aspect of our system (User Account, Recipe Exportation, Search, Pantry, User Favorites, and Custom Recipes) we will use a fluffer to determine what kind of Errors/Failures we can get out of it.

Additionally, we will be using a Trello Board connected to our Github to track the bugs that occur during testing. This will allow us to effectively identify, delegate, and prioritize fixes to our application.

Documentation plan

Help Menu's - We plan to have a help screen at the beginning of the user experience after they sign-in. This is to let them know how to use the system without being blind to different features.

Admin Guide - This will be available to admin that have access to the github repo. It will be under guide.md. This will allow any admins to quickly understand the system and how it is built.

Developer Documentation - We will ensure that the source code is well documented/commented, explaining what each function does and why it is important. This will ensure that anyone who is joining the team will be able to get working much faster.

Software Architecture

This project will be using a Client-Server architecture pattern. Below are more specifics about our planned architecture.

Major Components:

- Frontend (Done in React, hosted via Vercel)
    - Will contain the UI for the web applications features:
        - Login
        - Ingredient input
        - Search features
        - Checkout features
        - Recipe creation
    - Allows users to send requests to the backend
- Backend (Done in Node, hosted via Render)
    - Acts as the API layer
    - Retrieves data from MealDB and SupaBase
    - Formats DB searches
- User information db (Hosted on SupaBase)
    - Stores all user information: login info, saved recipes
    - Free cloud based persistent storage
- Recipe DB (Using the MEALDB)
    - An existing db of recipes that include ingredients and visuals

Interactions:

- The Frontend will interact with the backend. The users will be able to interact with an easy to understand interface, that will send JSON requests to the backend and then send JSON responses that will reflect in the frontend.
- The Backend will interact with both SupaBase and the MEALDB to get the requested information (Recipes, ingredients, user information). These interactions will take place through queries from the backend to the DBS.

Storage:

- We will be using SupaBase to store user information.\
- The information will be stored as the following:
    - Each user will have a unique ID
    - A username
    - Email
    - Password (To be encrypted)
    - An array of favorite meals

- The rest of the recipe storage will be used via the MEALDB. The names/ids of these recipes will be used for the array of favorites

Assumptions:

- The user will have a stable and reliable wifi connection to access our web app.
- The MEALDB will be up and running functionally.
- There are few users using our application at a time
    - Currently to test our product we will be using free versions of render and Vercel which will limit traffic.

Alternatives:

- **Decision 1:** The use of MongoDB for our user DB
    - **Pros:**
        - Works very well with Node
        - Some of the team members were already familiar with this software
        - Free version
        - Very flexible, allowing us to adapt the product as we progress and find we can add stretch features.
    - **Cons:**
        - High resource consumption
        - Potentially inconsistent
        - Can be slower then other options in some instances
    - **Alternative:** PostgreSQL
    - **Pros:**
        - Data is very secure
        - Much more robust features
        - Free
    - **Cons:**
        - More complex and difficult to learn
        - High memory usage
        - More overhead necessary

- **Decision 2:** Usage of a REST API
    - **Pros:**
        - Easy to implement in a small team
        - Language agnostic
        - Easy to debug
    - **Cons:**
        - Can over and/or under fetch data when making queries
        - Can lead to high network latency

    - **Alternative:** GraphQL
    - **Pros:**
        - Very flexible
        - Will only request necessary data
    - **Cons:**
        - Much more complicated
        - High overhead
        - Robustness can lead to more inefficiencies if not treated

---

Software Design

Frontend
The frontend will be responsible for the UI and handling user interactions. It will include the following:

- A page for login
- A page for search
- A page for users to view favorited recipes
- A page for users to checkout
- A page for users to input their ingredients

Additionally, the frontend will be responsible for handling http requests from users and sending those requests to the backend of the web application and then properly updating the state to reflect the output of these requests

Backend
The backend is responsible for handling query logic and will consist of an API layer, Request Handling Layer, an External API, and our own database layer.

The API Layer will contain the different routes that requests from the frontend can take. Currently, we plan on having a route for recipes and a route for users as those are the two kinds of data we will be storing.

The request Handling Layer will contain the code to complete these requests, such as searching for recipes or displaying recipes. This layer is responsible for sending the correct data to be displayed by the frontend.

The External API will be the MEALDB. We will be using many of the already existing functions with this database in order to display recipes and ingredients correctly.

Our personal database will be the database of Users done through SupaBase. This is responsible for storing users correctly and handling CRUD functions.

Coding Guideline

We will be guiding the development of our web application using the JavaScript coding conventions outlined by Google. We have chosen Google's guidelines because they are a reputable company known for their high quality web applications. These guidelines will be enforced via peer review, where we will read over each other's code and provide feedback or make corrections whenever we find code that does not align with our standards.

Google's JavaScript Guidelines:
https://google.github.io/styleguide/jsguide.html